Fast Path 순수 구현

🚀 Fast Path 핵심 개념

Fast Path는 Autobahn BFT의 핵심 성능 엔진으로, 네트워크 조건이 좋을 때 **2f+1 노드만으로 빠른 합의**를 달성합니다.

📊 Fast Path vs Slow Path 비교

| 특성 | Fast Path | Slow Path |

|------|----------|----------|

| **필요 노드 수** | 2f+1 | 3f+1 |

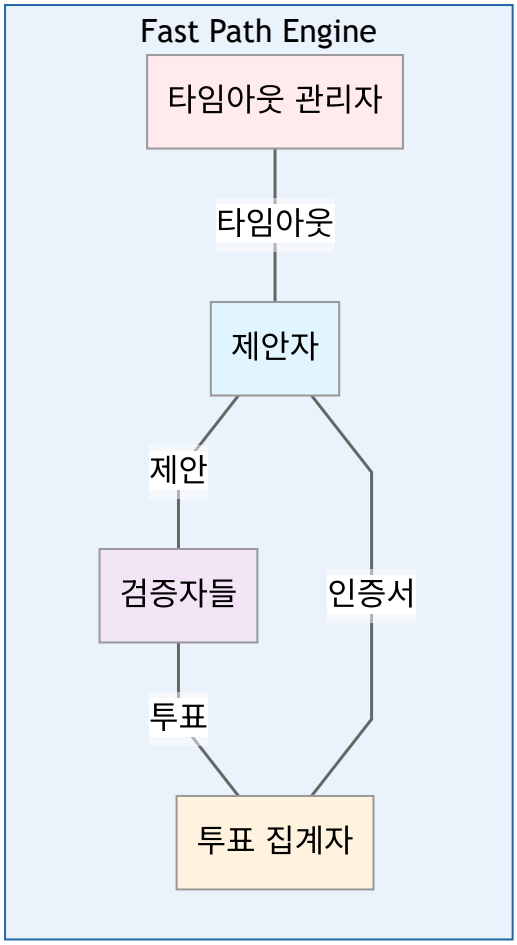
| **합의 단계** | 1단계 (투표) | 3단계 (Prepare/Promise/Commit) |

| **지연시간** | 50-120ms | 200-500ms |

| **처리량** | 80,000+ TPS | 30,000+ TPS |

| **네트워크 요구사항** | 안정적 | 불안정해도 동작 |

🏗️ Fast Path 아키텍처



🔧 핵심 컴포넌트 구현

1. Fast Path 제안자 (Proposer)

```
/// Fast Path 제안자 - 트랜잭션 배치를 제안
pub struct FastPathProposer {
/// 현재 라운드
current_round: AtomicU64,
/// 트랜잭션 풀
transaction_pool: Arc<RwLock<TransactionPool>>,
/// 배칭 최적화기
batch_optimizer: BatchOptimizer,
```

```
/// 암호학적 서명자
signer: CryptographicSigner,
}


impl FastPathProposer {
/// 새로운 Fast Path 제안 생성
pub  async  fn  create_proposal(&mut  self) -> Result<FastProposal, AutobahnErro
let  round = self.current_round.load(Ordering::SeqCst);
// 1. 최적 배치 크기 계산
let  batch_size = self.batch_optimizer.calculate_optimal_size().await;
// 2. 트랜잭션 선택
let  transactions = self.select_transactions(batch_size).await?;
// 3. 제안 생성
let  proposal = FastProposal {
round,
proposer: self.node_id(),
transactions,
timestamp: SystemTime::now(),
parent_hash: self.get_parent_hash().await?,
};
// 4. 암호학적 서명
let  signed_proposal = self.signer.sign_proposal(proposal)?;
// 5. 제안 검증
```

```
self.validate_proposal(&signed_proposal)?;

Ok(signed_proposal)

}

/// 트랜잭션 선택 알고리즘

async fn select_transactions(&self, batch_size: usize) -> Result<Vec<Transacti

let pool = self.transaction_pool.read().await;

// 우선순위 기반 선택

let mut selected = pool.get_highest_priority_transactions(batch_size * 2);

// 의존성 해결

selected = self.resolve_dependencies(selected)?;

// 최종 배치 크기로 조정

selected.truncate(batch_size);

Ok(selected)

}

/// 제안 검증

fn validate_proposal(&self, proposal: &FastProposal) -> Result<(), AutobahnErro

// 1. 라운드 검증

if proposal.round <= self.get_last_committed_round() {

return Err(AutobahnError::InvalidProposal("Round too old".into()));

}

// 2. 트랜잭션 검증

for tx in &proposal.transactions {

self.validate_transaction(tx)?;
```

```
}

// 3. 서명 검증

self.signer.verify_proposal_signature(proposal)?;

Ok(())

}

}
```

2. Fast Path 검증자 (Validator)

```
/// Fast Path 검증자 - 제안을 검증하고 투표

pub  struct FastPathValidator {

/// 노드 ID

node_id: NodeId,

/// 투표 생성기

vote_generator: VoteGenerator,

/// 제안 검증기

proposal_validator: ProposalValidator,

/// 중복 투표 방지

vote_tracker: VoteTracker,

}


impl FastPathValidator {
```

```
/// 제안에 대한 투표 생성

pub  async  fn  vote_on_proposal(&mut  self, proposal: FastProposal) -> Result<O

// 1. 중복 투표 체크

if  self.vote_tracker.has_voted_for_round(proposal.round) {

return Ok(None); // 이미 투표함

}

// 2. 제안 검증

let  validation_result = self.proposal_validator.validate(&proposal).await?;

match  validation_result {

ValidationResult::Valid => {

// 3. 찬성 투표 생성

let  vote = self.vote_generator.create_yes_vote(proposal).await?;

self.vote_tracker.record_vote(proposal.round, &vote);

Ok(Some(vote))

}

ValidationResult::Invalid(reason) => {

// 4. 반대 투표 생성

let  vote = self.vote_generator.create_no_vote(proposal, reason).await?;

self.vote_tracker.record_vote(proposal.round, &vote);

Ok(Some(vote))

}

ValidationResult::Abstain => {

// 5. 기권 (투표하지 않음)
```

```
Ok(None)
}
}
}
/// 제안 검증 로직
async fn validate_proposal_content(&self, proposal: &FastProposal) -> Result<V
// 1. 기본 구조 검증
if proposal.transactions.is_empty() {
return Ok(ValidationResult::Invalid("Empty transaction batch".into()));
}
// 2. 트랜잭션 개별 검증
for (i, tx) in proposal.transactions.iter().enumerate() {
match self.validate_transaction(tx).await {
Ok(true) => continue,
Ok(false) => return Ok(ValidationResult::Invalid(format!("Invalid transaction at
Err(e) => return Err(e),
}
}
// 3. 배치 일관성 검증
if !self.validate_batch_consistency(&proposal.transactions).await? {
return Ok(ValidationResult::Invalid("Batch inconsistency detected".into()));
}
// 4. 리소스 제한 검증
```

```
if !self.validate_resource_limits(proposal).await? {
return Ok(ValidationResult::Invalid("Resource limits exceeded".into()));
}
Ok(ValidationResult::Valid)
}
}

/// 투표 생성기
pub struct VoteGenerator {
/// 암호학적 서명자
signer: CryptographicSigner,
/// 노드 신원
node_identity: NodeIdentity,
}

impl VoteGenerator {
/// 찬성 투표 생성
pub async fn create_yes_vote(&self, proposal: FastProposal) -> Result<FastVot
let vote = FastVote {
round: proposal.round,
proposal_hash: proposal.hash(),
voter: self.node_identity.node_id(),
vote_type: VoteType::Yes,
```

```
timestamp: SystemTime::now(),

justification: None,

};

// 암호학적 서명 추가

let signed_vote = self.signer.sign_vote(vote)?;

Ok(signed_vote)

}

/// 반대 투표 생성

pub async fn create_no_vote(&self, proposal: FastProposal, reason: String) ->

let vote = FastVote {

round: proposal.round,

proposal_hash: proposal.hash(),

voter: self.node_identity.node_id(),

vote_type: VoteType::No,

timestamp: SystemTime::now(),

justification: Some(reason),

};

let signed_vote = self.signer.sign_vote(vote)?;

Ok(signed_vote)

}

}
```

3. Fast Path 투표 집계자 (Vote Aggregator)

```rust
/// Fast Path 투표 집계자 - 2f+1 투표로 인증서 생성
pub struct FastPathAggregator {
/// 라운드별 투표 저장소
votes_by_round: HashMap<u64, RoundVotes>,
/// 노드 집합 정보
node_set: NodeSet,
/// 임계값 계산기
threshold_calculator: ThresholdCalculator,
/// 인증서 생성기
certificate_generator: CertificateGenerator,
}

impl FastPathAggregator {
/// 투표 추가 및 집계
pub async fn add_vote(&mut self, vote: FastVote) -> Result<AggregationResult
// 1. 투표 검증
self.validate_vote(&vote)?;
// 2. 라운드별 투표 저장
let round_votes = self.votes_by_round.entry(vote.round).or_insert_with(RoundVot
// 3. 중복 투표 체크
if round_votes.has_vote_from(&vote.voter) {
return Ok(AggregationResult::Duplicate);
```

```
}

// 4. 투표 추가

round_votes.add_vote(vote.clone());

// 5. 임계값 체크

let  threshold = self.threshold_calculator.fast_path_threshold(); // 2f+1

if  round_votes.yes_votes_count() >= threshold {

// 6. 인증서 생성

let  certificate = self.certificate_generator.create_fast_certificate(

vote.round,

vote.proposal_hash,

round_votes.get_yes_votes(),

)?;

return Ok(AggregationResult::Success(certificate));

}

// 7. 실패 조건 체크

if  round_votes.no_votes_count() > self.node_set.size() - threshold {

return Ok(AggregationResult::Failed);

}

Ok(AggregationResult::Continue)

}

/// 투표 검증

fn  validate_vote(&self, vote: &FastVote) -> Result<(), AutobahnError> {

// 1. 서명 검증
```

```
if !self.verify_vote_signature(vote)? {
return Err(AutobahnError::InvalidVote("Invalid signature".into()));
}
// 2. 투표자 검증
if !self.node_set.contains(&vote.voter) {
return Err(AutobahnError::InvalidVote("Unknown voter".into()));
}
// 3. 타임스탬프 검증
if  self.is_vote_too_old(vote) || self.is_vote_too_future(vote) {
return Err(AutobahnError::InvalidVote("Invalid timestamp".into()));
}
Ok(())
}
}


/// 라운드별 투표 관리
pub  struct RoundVotes {
/// 찬성 투표들
yes_votes: Vec<FastVote>,
/// 반대 투표들
no_votes: Vec<FastVote>,
/// 투표자 집합 (중복 방지)
voters: HashSet<NodeId>,
```

```
}

impl RoundVotes {
pub fn new() -> Self {
Self {
yes_votes: Vec::new(),
no_votes: Vec::new(),
voters: HashSet::new(),
}
}
pub fn add_vote(&mut self, vote: FastVote) {
if self.voters.insert(vote.voter.clone()) {
match vote.vote_type {
VoteType::Yes => self.yes_votes.push(vote),
VoteType::No => self.no_votes.push(vote),
}
}
}
pub fn yes_votes_count(&self) -> usize {
self.yes_votes.len()
}
pub fn no_votes_count(&self) -> usize {
self.no_votes.len()
```

```
}
pub fn has_vote_from(&self, voter: &NodeId) -> bool {
self.voters.contains(voter)
}
pub fn get_yes_votes(&self) -> &[FastVote] {
&self.yes_votes
}
}
```

4. 적응형 타임아웃 관리자

```
/// 적응형 타임아웃 관리자 - 네트워크 조건에 따른 동적 타임아웃
pub struct AdaptiveTimeoutManager {
/// 기본 타임아웃
base_timeout: Duration,
/// 네트워크 지연 히스토리
latency_history: CircularBuffer<Duration>,
/// 성공률 히스토리
success_rate_history: CircularBuffer<f64>,
/// 현재 타임아웃
current_timeout: Duration,
/// 조정 파라미터
adjustment_params: TimeoutAdjustmentParams,
```

```
}

impl AdaptiveTimeoutManager {
/// 새로운 타임아웃 계산
pub fn calculate_adaptive_timeout(&mut self) -> Duration {
// 1. 평균 지연시간 계산
let avg_latency = self.latency_history.average();
// 2. 지연시간 분산 계산
let latency_variance = self.latency_history.variance();
// 3. 성공률 계산
let success_rate = self.success_rate_history.average();
// 4. 적응형 타임아웃 공식
// timeout = base + α * avg_latency + β * sqrt(variance) + γ * (1 - success_rate
let adaptive_component =
self.adjustment_params.latency_factor * avg_latency.as_secs_f64() +
self.adjustment_params.variance_factor * latency_variance.sqrt() +
self.adjustment_params.failure_factor * (1.0 - success_rate);
let new_timeout = self.base_timeout + Duration::from_secs_f64(adaptive_componen
// 5. 최소/최대 제한 적용
let bounded_timeout = new_timeout
.max(self.adjustment_params.min_timeout)
.min(self.adjustment_params.max_timeout);
self.current_timeout = bounded_timeout;
```

```
bounded_timeout

}

/// 라운드 결과 기록

pub fn record_round_result(&mut self, latency: Duration, success: bool) {

self.latency_history.push(latency);

self.success_rate_history.push(if success { 1.0 } else { 0.0 });

}

/// 네트워크 조건 변화 감지

pub fn detect_network_change(&self) -> NetworkConditionChange {

let recent_latency = self.latency_history.recent_average(10);

let historical_latency = self.latency_history.average();

let recent_success = self.success_rate_history.recent_average(10);

let historical_success = self.success_rate_history.average();

if recent_latency > historical_latency * 1.5 || recent_success < historical_suc

NetworkConditionChange::Degraded

} else if recent_latency < historical_latency * 0.7 && recent_success > histor

NetworkConditionChange::Improved

} else {

NetworkConditionChange::Stable

}

}

}
```

```rust
/// 타임아웃 조정 파라미터
#[derive(Debug, Clone)]
pub struct TimeoutAdjustmentParams {
/// 지연시간 가중치
pub latency_factor: f64,
/// 분산 가중치
pub variance_factor: f64,
/// 실패율 가중치
pub failure_factor: f64,
/// 최소 타임아웃
pub min_timeout: Duration,
/// 최대 타임아웃
pub max_timeout: Duration,
}

impl Default for TimeoutAdjustmentParams {
fn default() -> Self {
Self {
latency_factor: 2.0,
variance_factor: 1.0,
failure_factor: 0.5,
min_timeout: Duration::from_millis(50),
max_timeout: Duration::from_secs(5),
```

```
}

}

}
```

🔄 Fast Path 실행 흐름

```
/// Fast Path 전체 실행 흐름

pub struct FastPathExecutor {

proposer: FastPathProposer,

validator: FastPathValidator,

aggregator: FastPathAggregator,

timeout_manager: AdaptiveTimeoutManager,

}


impl FastPathExecutor {

/// Fast Path 라운드 실행

pub async fn execute_round(&mut self) -> Result<FastPathResult, AutobahnErro

let round_start = Instant::now();

// 1. 제안 생성 (리더인 경우)

if self.is_leader_for_current_round() {

let proposal = self.proposer.create_proposal().await?;

self.broadcast_proposal(proposal.clone()).await?;
```

```
}
// 2. 제안 수신 대기
let proposal = self.wait_for_proposal().await?;
// 3. 투표 생성
if let Some(vote) = self.validator.vote_on_proposal(proposal).await? {
self.broadcast_vote(vote).await?;
}
// 4. 투표 수집 및 집계
let aggregation_result = self.collect_and_aggregate_votes().await?;
// 5. 결과 처리
let round_latency = round_start.elapsed();
match aggregation_result {
AggregationResult::Success(certificate) => {
self.timeout_manager.record_round_result(round_latency, true);
Ok(FastPathResult::Success(certificate))
}
AggregationResult::Failed => {
self.timeout_manager.record_round_result(round_latency, false);
Ok(FastPathResult::Failed)
}
AggregationResult::Timeout => {
self.timeout_manager.record_round_result(round_latency, false);
Ok(FastPathResult::Timeout)
```

```
}
}
}
/// 투표 수집 및 집계
async fn collect_and_aggregate_votes(&mut self) -> Result<AggregationResult,
let timeout = self.timeout_manager.calculate_adaptive_timeout();
let deadline = Instant::now() + timeout;
while Instant::now() < deadline {
// 투표 수신
if let Some(vote) = self.receive_vote_with_timeout(Duration::from_millis(100)).
// 투표 집계
match self.aggregator.add_vote(vote).await? {
AggregationResult::Success(cert) => return Ok(AggregationResult::Success(cert)),
AggregationResult::Failed => return Ok(AggregationResult::Failed),
AggregationResult::Continue => continue,
AggregationResult::Duplicate => continue,
}
}
}
Ok(AggregationResult::Timeout)
}
}
```

📊 성능 최적화

1. 배칭 최적화

```
/// 배칭 최적화기 - 네트워크 조건에 따른 동적 배치 크기 조정
pub struct BatchOptimizer {
/// 현재 배치 크기
current_batch_size: usize,
/// 처리량 히스토리
throughput_history: CircularBuffer<f64>,
/// 지연시간 히스토리
latency_history: CircularBuffer<Duration>,
/// 최적화 파라미터
optimization_params: BatchOptimizationParams,
}

impl BatchOptimizer {
/// 최적 배치 크기 계산
pub async fn calculate_optimal_size(&mut self) -> usize {
let current_throughput = self.throughput_history.recent_average(5);
let current_latency = self.latency_history.recent_average(5);
// 처리량 우선 모드
if self.optimization_params.prioritize_throughput {
```

```
if current_throughput < self.optimization_params.target_throughput {
self.current_batch_size = (self.current_batch_size * 110 / 100)
.min(self.optimization_params.max_batch_size);
}
}
// 지연시간 우선 모드
else {
if current_latency > self.optimization_params.target_latency {
self.current_batch_size = (self.current_batch_size * 90 / 100)
.max(self.optimization_params.min_batch_size);
}
}
self.current_batch_size
}
}
```

2. 병렬 처리 최적화

```
/// 병렬 처리 최적화 - 투표 검증 및 집계 병렬화
pub struct ParallelVoteProcessor {
/// 워커 스레드 풀
thread_pool: ThreadPool,
/// 검증 큐
```

```
validation_queue: Arc<Mutex<VecDeque<FastVote>>>,
/// 집계 큐
aggregation_queue: Arc<Mutex<VecDeque<ValidatedVote>>>,
}

impl ParallelVoteProcessor {
/// 병렬 투표 처리
pub  async  fn  process_votes_parallel(&self, votes: Vec<FastVote>) -> Result<Ve
let (tx, rx) = mpsc::channel();
// 투표들을 워커 스레드들에 분배
for  vote  in  votes {
let  tx_clone = tx.clone();
self.thread_pool.execute(move || {
let  result = Self::validate_vote_sync(vote);
tx_clone.send(result).unwrap();
});
}
// 결과 수집
let  mut  validated_votes = Vec::new();
for  _  in  0..votes.len() {
if  let Ok(validated_vote) = rx.recv() {
validated_votes.push(validated_vote?);
}
```

```
}

Ok(validated_votes)

}

}
```

🎯 Fast Path 성능 특성

예상 성능 지표

```
처리량: 80,000+ TPS

지연시간: 50-120ms (네트워크 조건에 따라)

CPU 사용률: 60-80%

메모리 사용량: 1-2GB (노드당)

네트워크 대역폭: 100-500 Mbps
```

확장성

```
노드 수: 최대 500+ 노드 (Fast Path)

지리적 분산: 대륙간 지원

처리량 확장: 노드 수에 비례하여 선형 증가
```

🔍 모니터링 및 디버깅

```rust
/// Fast Path 성능 모니터링
pub struct FastPathMonitor {
/// 메트릭 수집기
metrics_collector: MetricsCollector,
/// 성능 분석기
performance_analyzer: PerformanceAnalyzer,
}

impl FastPathMonitor {
/// 실시간 성능 모니터링
pub async fn monitor_performance(&mut self) -> FastPathMetrics {
FastPathMetrics {
current_round: self.get_current_round(),
rounds_per_second: self.calculate_rounds_per_second(),
average_latency: self.calculate_average_latency(),
success_rate: self.calculate_success_rate(),
timeout_rate: self.calculate_timeout_rate(),
vote_aggregation_time: self.measure_aggregation_time(),
}
}
}
```

🎉 결론

Pure Fast Path 구현은:

- ✅ **최고 성능**: 2f+1 노드로 최소 오버헤드
- ✅ **적응형 최적화**: 네트워크 조건에 따른 동적 조정
- ✅ **병렬 처리**: 투표 검증 및 집계 병렬화
- ✅ **실시간 모니터링**: 성능 지표 실시간 추적

다음 문서에서는 Slow Path의 순수 구현에 대해 알아보겠습니다.